

Debugowanie Windows – przygotowanie środowiska

Autorzy: Bogusław Okuniewski, Tomasz Dziubich

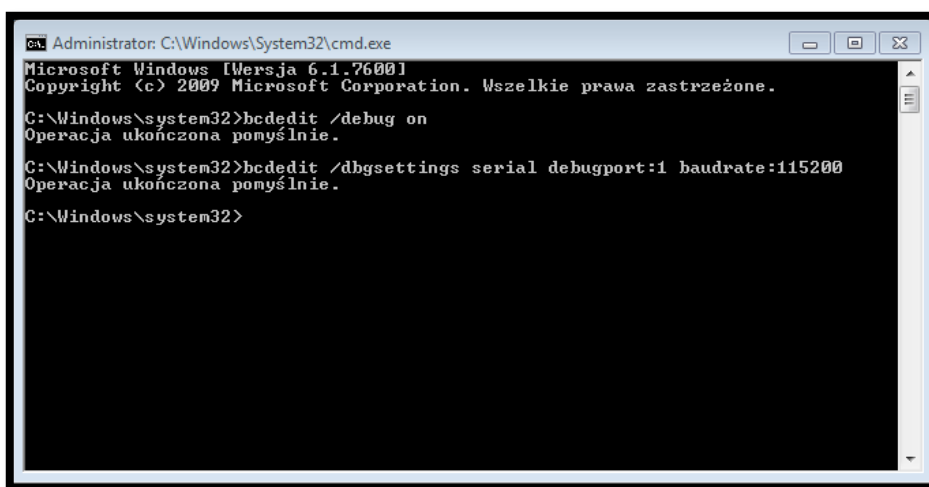
System host: Windows 7 x64, sprawdzone również na Windows Server 2008

System guest: Windows 7 x64 postawiony na VirtualBox

1. Uruchomić system na wirtualnej maszynie, otworzyć konsolę jako administrator i edytować plik „BOOT.ini” poprzez wpisanie:

bcdedit /debug on

bcdedit /dbgsettings serial debugport:1 baudrate:115200



```
Administrator: C:\Windows\System32\cmd.exe
Microsoft Windows [Wersja 6.1.7600]
Copyright (c) 2009 Microsoft Corporation. Wszelkie prawa zastrzeżone.

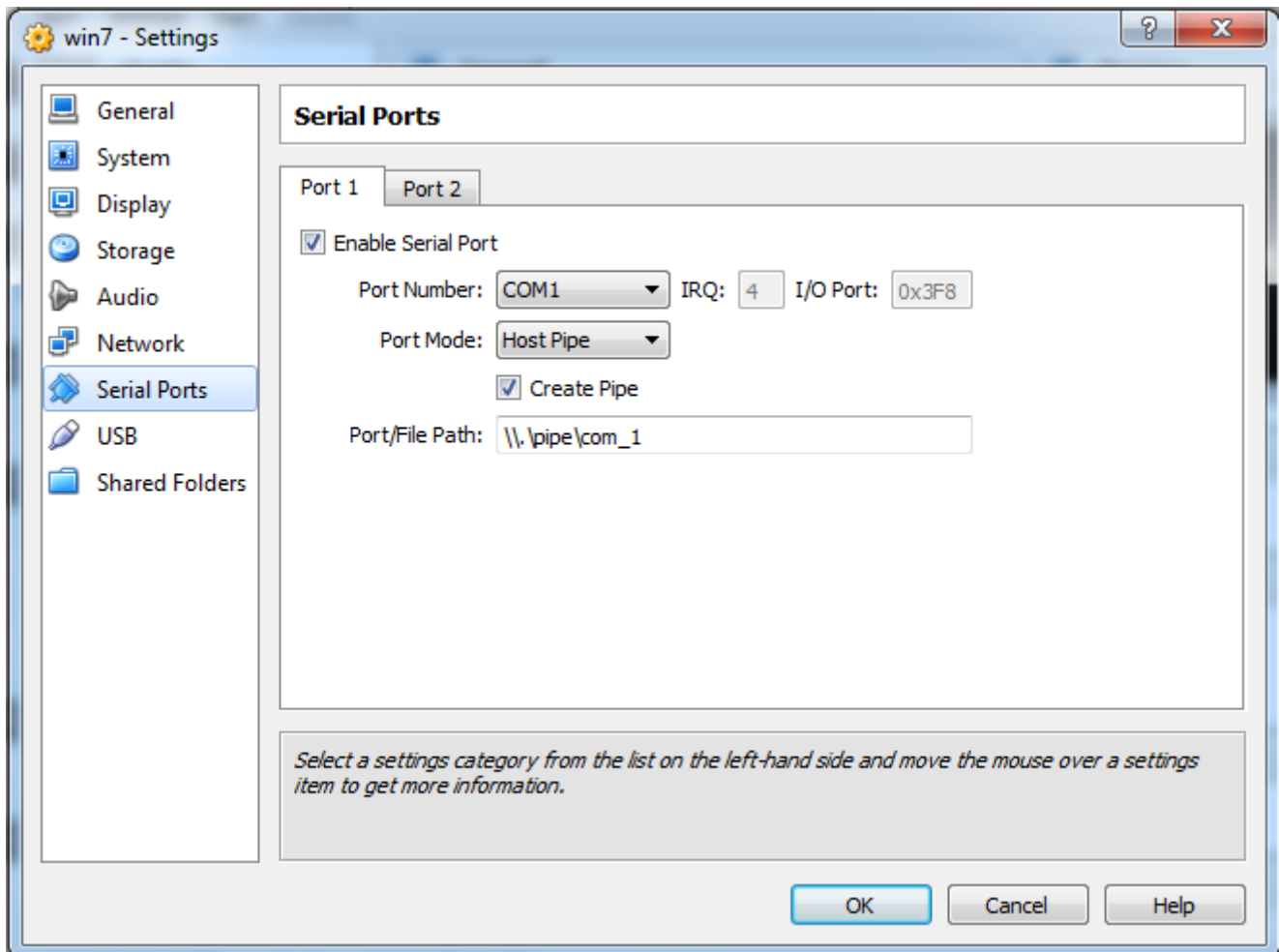
C:\Windows\system32>bcdedit /debug on
Operacja ukończona pomyślnie.

C:\Windows\system32>bcdedit /dbgsettings serial debugport:1 baudrate:115200
Operacja ukończona pomyślnie.

C:\Windows\system32>
```

Należy pamiętać, że polecenie **bcdedit** występuje dopiero od Windows Vista. Dla starszych wersji systemów musimy ręcznie edytować plik „BOOT.ini”.

2. Uruchamiamy wirtualną maszynę.
3. Wejść w ustawienia naszej wirtualnej maszyny w VirtualBoxie, następnie wybieramy z prawego menu „Serial Ports”. W zakładce „Port 1” zaznaczamy „Enable Serial Port”. Wybieramy „Port number: COM1”, „IRQ” i „I/O Port” zostawiamy domyślne. „Port Mode” wybieramy: „Host Pipe”. Zaznaczamy „Create Pipe”. W pole „Port/File Path” wpisujemy: `\\.\pipe\com_1`



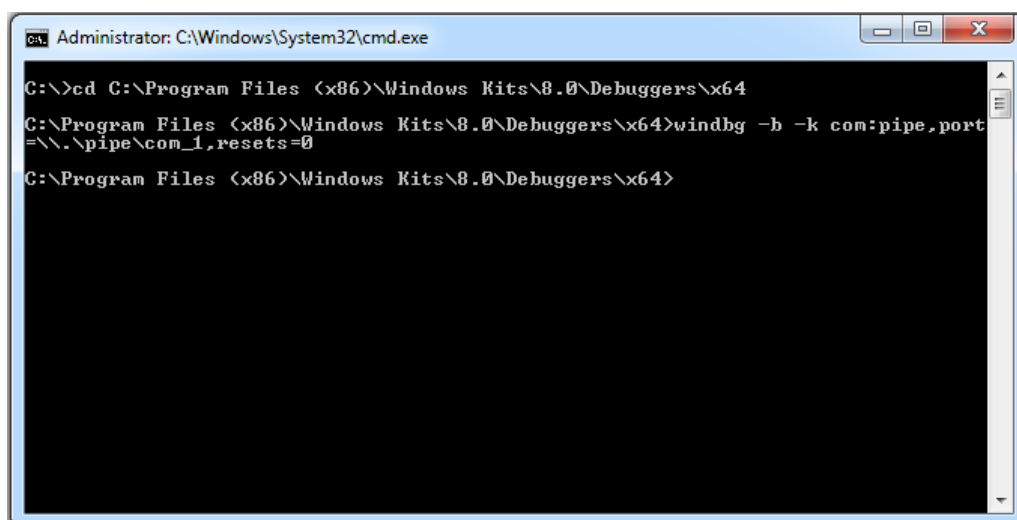
Wirtualna maszyna jest ustawiona.

4. Na hoście instalujemy Debugging Tools for Windows:

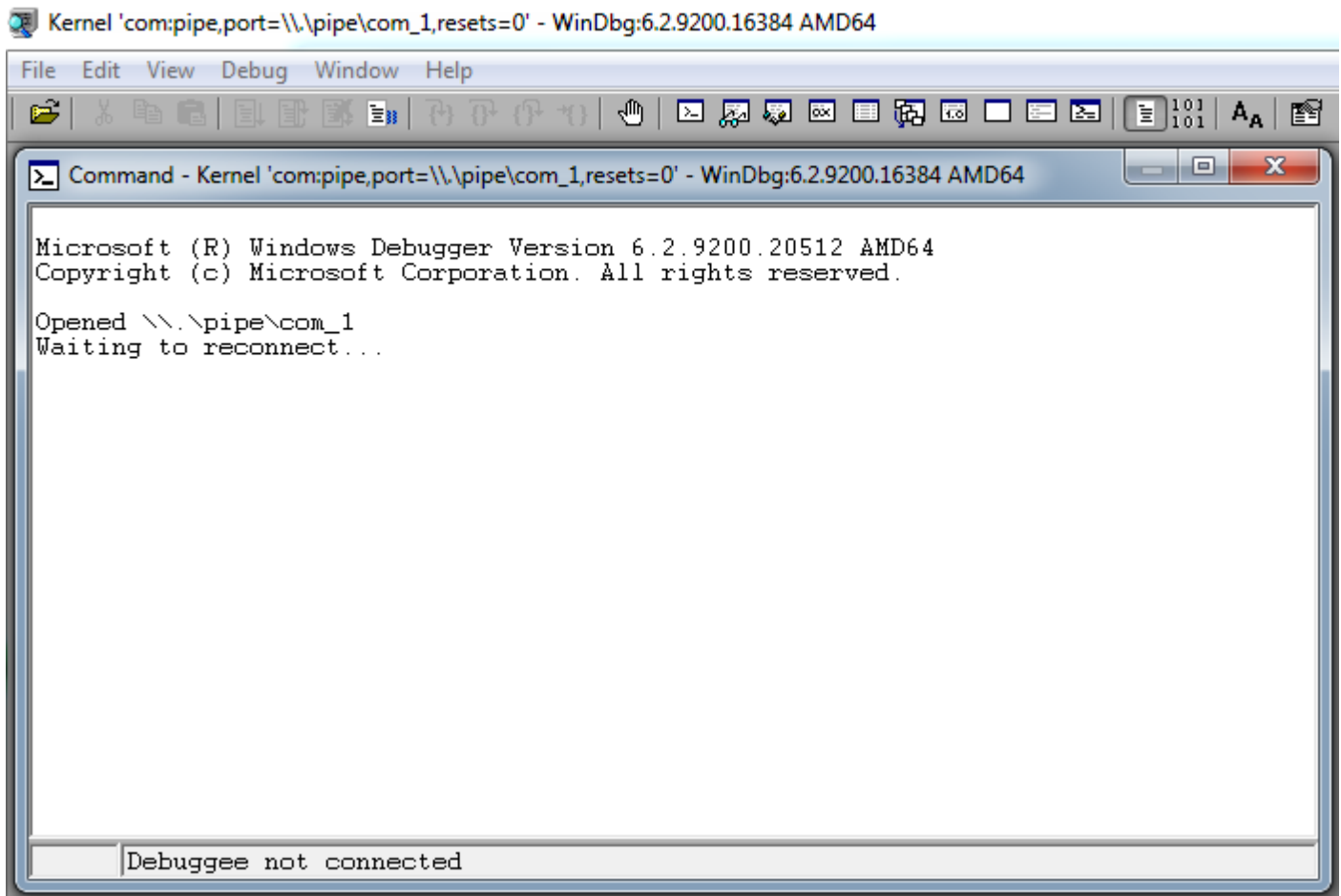
<http://msdn.microsoft.com/pl-pl/windows/hardware/hh852363>

5. Uruchamiamy naszą maszynę wirtualną, na hoście odpalamy konsolę jako administrator i przechodzimy do miejsca instalacji (tam gdzie znajduje się „windbg.exe”) i wpisujemy:

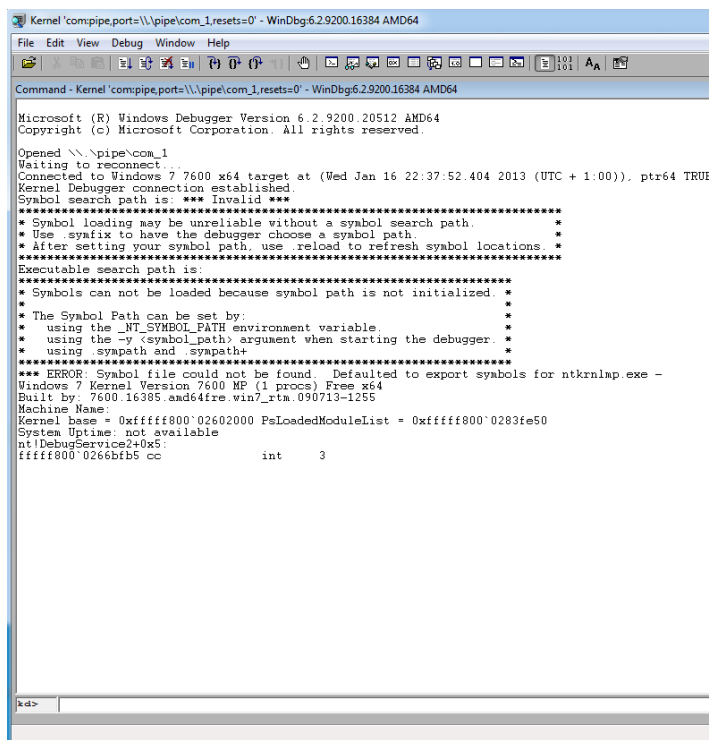
windbg -b -k com:pipe,port=\\.\pipe\com_1, resets=0



6. Powinno pokazać się nam następujące okno z komunikatem:



7. Uruchamiamy ponownie naszą wirtualną maszynę, jeżeli wszystko dobrze zrobiliśmy to podczas ładowania systemu zostanie wstrzymana, a na hoście w okienku pojawi się informacja o nawiązaniu połączenia:



Oprócz tego otrzymujemy błąd odnośnie symboli:

***** ERROR: Symbol file could not be found.
Defaulted to export symbols for
ntkrnlmp.exe -**

8. Naprawiamy problem z symbolami. Wpisujemy komendę:

.sympath SRV*c:\Temp\DebugSymbols*http://msdl.microsoft.com/download/symbols

Komenda zakłada, że nasze symbole znajdują się w **D:\DebugSymbols**

Następnie ładujemy nasze symbole komendą:

.reload /f

9. Środowisko jest teraz gotowe do debugowania sytemu i aplikacji.

10. Breakpointy stawiamy w następujący sposób:

kd> bu module!name

Jeżeli nasz sterownik nazywa się „**Test.sys**” i chcemy go zatrzymać na funkcji wejściowej DriverEntry() robimy to tak:

kd> bu Test!DriverEntry

11. Ładujemy na maszynie wirtualnej nasz sterownik (plik.sys). Po załadowaniu powinno nastąpić ponowne zatrzymanie, ponieważ osiągnęliśmy break point.

12. Przydatne komendy kd poniżej. Opis wszystkich komend można znaleźć pod <http://msdn.microsoft.com/en-us/library/windows/hardware/ff539170%28v=vs.85%29.aspx>. Bardziej czytelny pod <http://www.windbg.info/doc/1-common-cmds.html>.

Breakpoints:	
bl	List of breakpoints
be	Breakpoint enable
bd	Breakpoint disable
bc	Breakpoint clear
bp Address	Set breakpoint
bu <ModuleName>!<FunctionName>	Set unresolved breakpoint. Breakpoint is resolved by symbolic name, not absolute address. Use this to set breakpoint at a function whose containing module has not yet been loaded.

Disassemble:	
u	Unassemble next few instructions
u [Address]	Unassemble instructions at Address
u [Range]	Unassemble in selcted Range

Stack trace:	
k [Number]	Displays stack trace of current thread (number of frames).
kb [Number]	Kb causes the display to include the first three parameters passed to each function.

Register watch:	
r [RegisterName]	Displays register

Continue:	
g [Address Symbol]	Go. Resumes execution until Address/Symbol
gh	Go, exception handled
Gn	Go, exception not handled

Quit	
Q	quit

Dumping data:	
D [b w d u a s S]	Display memory: byte word dword Unicode chars ASCII string unicode string
dv	Display local variables (You need private symbols)
dt Address[Type][Options]	Dump type; Options=[-b display substructures -r[depth] full recursive]

Edit Values:	
e[b d D f p a u]	Edit memory; byte dword double float pointer ascii unicode

List modules:	
lm	List loaded modules

Threads	
~	Lists all threads
~n<CommandName>	Switch to a specific thread by thread-id and execute a command on the thread.